

CS-UY 3923/CS-GY 6813: Information Security & Privacy

Evan Johnson, Fall 2026, Lecture 1

Introduction, Threat-Modeling, Tools of the Trade

Who am I?

Assistant professor in Computer Security ([website](#))

- Building secure systems
- Programming language techniques for security
- Formal verification for security

Tell me about yourselves!

- Taken OS? Architecture?
- Familiar with Unix/Linux tools? C++? GDB?

Contacts

- Professor: Evan Johnson
 - Lectures: Tuesday 6:30:-9:00 (here)
 - Office Hours: 2:30-3:30pm Thursdays @1055 370 Jay St
- Class Assistants
 - Aryan Chaudhary: 4-5pm Tuesdays (location TBD)
 - Will Liu: 1-2pm Mondays (1068 370 Jay St)
- [Brightspace](#)



Course tools / materials

- [Brightspace](#)
- Syllabus (posted on brightspace)
- Gradescope for assignment submission (through brightspace)
- No textbook, but optional readings (both academic and... not)
- Slides will be made available online
 - Based on notes from Shravan Narayan, Justin Cappos, Matthias Payer, Stefan Savage, Deepak Kumar, and Deian Stefan

Attention: this course is in development



Pros:

- More deep technical content (more reading code/ writing exploits, etc)
- If I mess up logistics, I will err on the side of giving you better grades

Cons:

- More than average change to course schedule and policies (based on feedback/difficulty etc.)

What you need

Laptop/Desktop that can build and run POSIX tools / qemu emulator

- Use Linux. I / TA can't help you debug everyone's Mac/Windows (WSL) setup
- But you can try:
 - **On Windows:** either install Linux in parallel or run qemu
 - **On Mac:** use Asahi Linux or parallels

What you will need to know (for projects)

- C/C++ programming and bits of assembly
- A bit of OS (memory protection, address translation, threads)
- Some architecture (caches/TLBs)
- I'll try to explain the key ideas when these things come up, but be prepared to learn on your own

Tentative Grading

- Project (5 parts): 25%
- Midterms (2): 40%
- Final: 35%

Rules

- 3 late days across all project checkpoints
 - Use at your own discretion, no email needed
- May work in groups of 2 or individually
- Regrades should be the exception
 - We reserve the right to completely regrade your assignments
- **No Cheating**
 - Read and understand [NYU's Academic Integrity Policy](#)
 - On programming assignments, you may use AI, if you cite how you used it.
 - “To write code”, “to explain concepts/code”, “to brainstorm”, etc
 - No AI will be permitted on exams (they will be pen and paper)
 - Not ok to copy, translate, paraphrase, edit, etc. someone else's work

Rules

- 3 late days across all project checkpoints
 - Use at your own discretion, no email needed
- May work in groups of 2 or individually
- Regrades should be the exception
 - We reserve the right to completely regrade your assignments
- **No Cheating**
 - Read and understand [NYU's Academic Integrity Policy](#)
 - On programming assignments, you may use AI, if you cite how you used it.
 - “To write code”, “to explain concepts/code”, “to brainstorm”, etc
 - No AI will be permitted on exams (they will be pen and paper)
 - Not ok to copy, translate, paraphrase, edit, etc. someone else's work

Ethics

- In this class you will learn how to attack computer systems
- We learn attacks because it is needed to understand how to defend
- You have an obligation to use this knowledge ethically
 - You **may not** attack others
 - In addition to being unethical, it may also be a felony
 - Many good **legitimate** hacking challenges
 - <http://overthewire.org/wargames/> (wargames)
 - <https://challenges.re/> (reverse engineering challenges)
 - <https://ctftime.org/ctfs> (Capture the Flag competitions)

A note on lecture length

- I'm aware that 2.5 hours is quite long for a lecture
- What I will do to help
 - Provide 1-2 5 minute intermissions throughout the class
 - Provide lots of opportunities to participate in class
- What you can do to help
 - Participate in class (it will keep you awake!)

Course Objectives

- A solid foundation of security concepts, backed by concrete examples
- Security mindset
 - How to **think like** an attacker/ security engineer
 - Looking beyond the system's intended functionality, to what it can be made to do
- Understanding how things work, how they break, and how to fix them
 - Technical details of vulnerabilities, attacks, and defenses
- Becoming a better engineer
 - Minimize number and severity of *vulnerabilities you create*
 - Understand the causes and impact of *vulnerabilities that you are alerted to*
 - Properly address *vulnerabilities that are identified*

The Plan

- What is computer security?
- Threat modeling & safety policies
- x86, C, and GDB

The Plan

- What is computer security?
- Threat modeling & safety policies
- x86, C, and GDB

What is computer security?

- Most of computer science is about making things happen:
 - UX/UI
 - Software Architecture
 - Algorithms
 - Operating Systems/Networking/Databases
 - Compilers/PL
 - Microarchitecture
 - VLSI/CAD
-
-
-

What is computer security?

- Most of computer science is about making things happen:
 - UX/UI
 - Software Architecture
 - Algorithms
 - Operating Systems/Networking/Databases
 - Compilers/PL
 - Microarchitecture
 - VLSI/CAD
- Computer security is about making things not happen
- Inherently cross-cutting
- It is about how some implementation of functionality behaves ***in the presence of an adversary***

Why understanding security matters

All developers should have some understanding of security

- Design systems ahead of time to be resilient

Unfortunately, this is not the case today

- Systems are often hurriedly written with no focus on security
- **Goal:** ship a “working” MVP
- **Result:** your system works, but gets hacked sooner or later

This class: thinking like an adversary and thinking like a defender

- **Note:** this is a lens/mindset, not a CS-specific skill

Thinking like an attacker

- Look for the weakest links
- Identify the **assumptions** that security depends on. Are they guaranteed to always be true? Can you make them false?
- Think outside the box. Ignore the limited worldview of the system's designers
- This is something you can start doing now and all the time. When you interact with a system (computerized or not) think about what that system depends on and how it might be exploited

Thinking like an attacker

What assumptions did the designers make?

How can we bypass them?



Thinking like an attacker



Thinking like an attacker

What assumptions did the designers make?

How can we bypass them?



Thinking like an attacker



History: two competing philosophies

- **Binary** model [secure vs insecure]
 - Traditional crypto and trustworthy systems
 - Assume adversary limitations X and define security policy Y
 - If Y cannot be violated without needing X then system is secure, else insecure
 - You know people are invoking some version of this model if they say “proof of security”, “secure by design” or “trustworthy systems”
- **Risk management** model. [more secure vs less secure]
 - Most commercial software development (and much real-world security... e.g., terrorism)
 - Try to identify most significant risks to security and minimize them
 - Improve security where most cost effective (expected value)
 - You know people are in this model if they use the words “risk”, “mitigation”, “defenses”, “resilience”, etc.

Classic example (binary model): perfect substitution cipher

$$\begin{array}{ccccccc} & p_1 & p_2 & p_3 & \dots & p_n & \\ \oplus & b_1 & b_2 & b_3 & \dots & b_n & \\ \hline c_1 & c_2 & c_3 & \dots & c_n & \end{array}$$

- Invented by combination of Vernam & Mauborgne (~1919)
- Choose a string of **random** bits the same length as the plaintext (key), XOR them with plaintext to obtain the ciphertext
 - Assume key is only known to sender and receiver
- **Perfect Secrecy** (proved by Claude Shannon)
 - Probability that a given message is encoded in the ciphertext is **unaltered** by knowledge of the ciphertext
 - Proof: Give me any plaintext message and any ciphertext and I can construct a key that will produce the ciphertext from the plaintext.
Zero information in ciphertext

Classic example (risk management): Concrete barricades

- Prevent incursion by car bombers



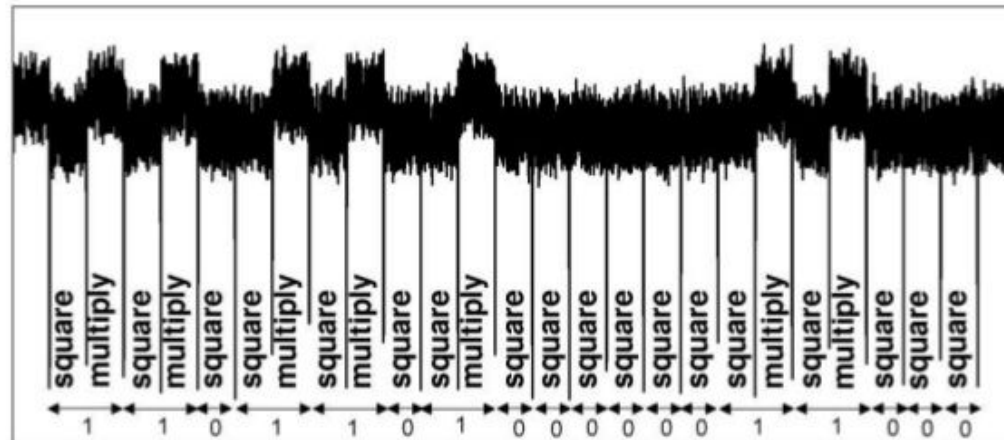
Binary security vs Risk Management

Other examples?

Problems w/ binary model: Abstract design != Concrete artifact

Many assumptions are **brittle** in real systems

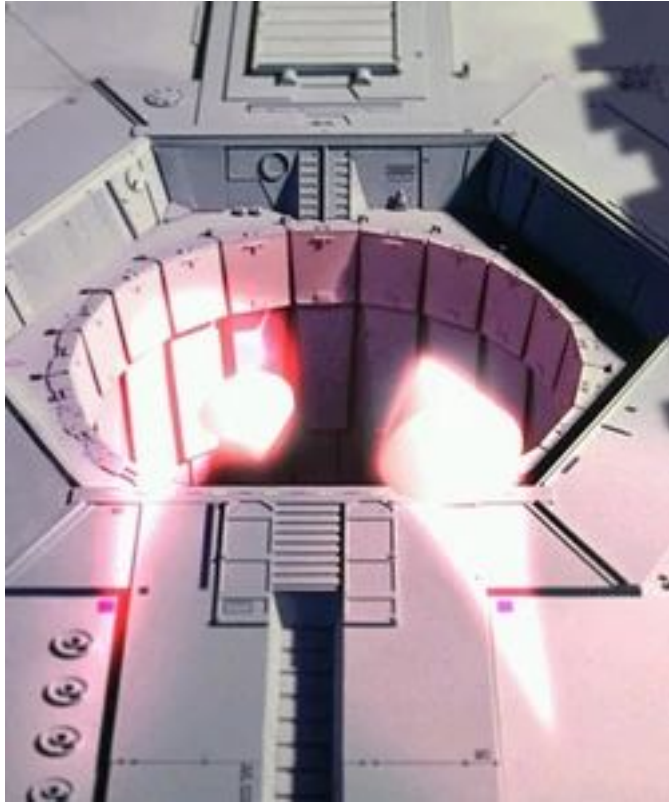
- Real artifacts fragile, imperfect, have bugs/limitations
 - Don't do precisely what spec says or documentation says
 - E.g., what is an integer?
- Large gap between abstraction and implementation
 - Example: secret key in chip used to decrypt data; key leaks via the current the chip draws for different operations



Problems w/ the binary model: security evolution

- As engineers, we often delude ourselves into thinking that we understand our own creations
- But ... **nobody knows how these systems really work**
 - Complexity of computer systems is approaching complexity of biological organisms
 - 3 billion base pairs in human genome
 - 10+ billion transistors in modern CPUs
 - And that's even before LLMs and modern AI...
- Complex systems co-evolve with attacks against them
 - How we use systems and how they might be attacked change over time
 - Systems deemed secure today may not be resilient to new threats

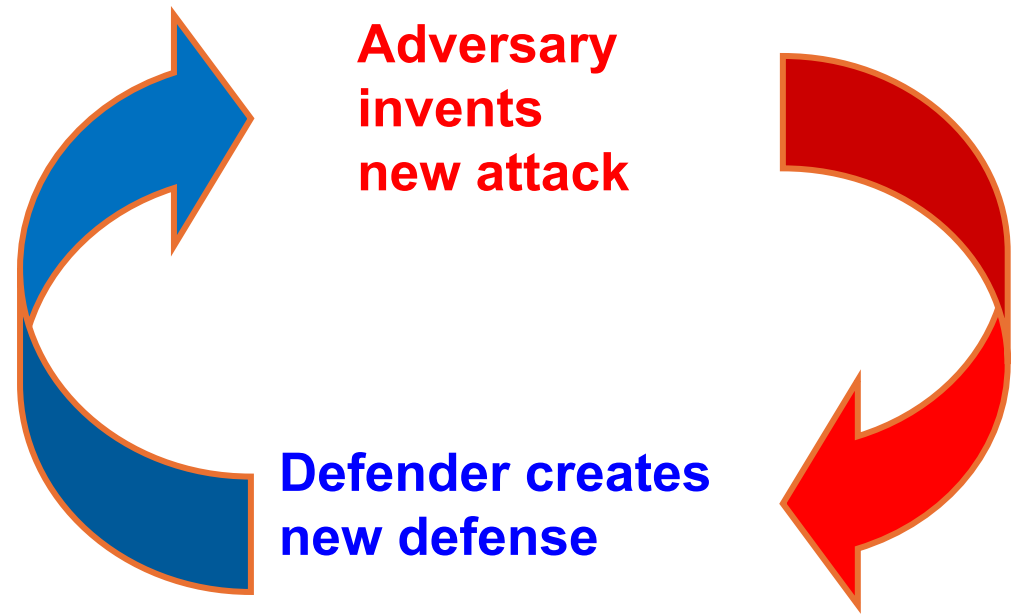
Problems w/ risk management model:
One vulnerability can matter...



Problems w/ risk management model: You never win

- Creates arms race – forced co-evolution

- Can just produce **stalemate**



Problems w/ risk management model: How to measure?

- It's fine to say security is a spectrum, but how to **evaluate** risk or reward?
 - How many **units of security** does your anti-virus give you?
- Big question: how do we measure security?
 - How is this different from car safety?
 - Or drug safety?

The Plan

- What is computer security?
- Threat modeling & safety policies
- x86, C, and GDB

Threat Model

What are we trying to protect from who?

Threat Model



What are we trying to protect from who?

The diagram consists of two overlapping ovals. The left oval is green and the right oval is red. The text 'What are we trying to protect from who?' is centered within the intersection of these two ovals.

Safety Policy

Attacker model

Safety Policy

- What are we trying to avoid?
- Remarkably tricky to define for known threats
 - What is the right security policy for a Web browser?
 - It has many dozens of security configuration options... How should you set them?
 - What might be a good security policy for who gets to access employee salary data?
- Even harder for unknown threats
 - SPAM – **who** should be allowed to send **you** e-mail?

Safety Policy

Other examples of safety policies that you might want to enforce?

Confidentiality

- Who can read what?
- Examples
 - Someone reading your private e-mail
 - An untrusted process can read your crypto key from memory
- What are some other examples?

Integrity

- Who can write what?
- Examples
 - Increasing your bank account balance without depositing money
 - Getting snacks from vending machine without paying for them
- What are some other examples?

Availability

- How can users affect each others service
 - Unauthorized parties can't prevent other users from accessing system
- Examples
 - Putting superglue in the ATM card slot
 - Network denial of service attacks (e.g., packet floods)
- What are other examples of denial of service?

C.I.A.

- Which security property is violated if someone ...
 - Unplugs your alarm clock while you're sleeping?
 - Changes the time on your alarm clock?
 - Retrieves your alarm history by hacking your SmartAlarm™ device

Vulnerabilities

- Weaknesses that could be exploited to cause damage to assets (typically where CIA are not enforced as intended)
 - Default password left intact (“password”)
 - Implementation flaws in software
 - Debug interface left open to network
- In particular, look to ***assumptions*** in system
- Known vulnerabilities (e.g., documented in public forums, NVD) vs unknown vulnerabilities (0-days)

Protection Mechanisms

- The mechanisms used to protect resources against threats
 - This is most of academic and industrial computer security
- Many classes of protections
 - Cryptographic protection of data
 - Software guards
 - Communication guards
 - User interface design (protect user against own limitations)
- Can be either proactive or reactive

Attacker model

Who is your attacker?

- Are they a disgruntled customer looking for revenge?
 - Your companies VP trying to embezzle money?
 - A curious classmate looking over your shoulder?
 - The NSA?
-
- Who your attacker is defines their capabilities
 - Are they just attacking your public website, or internal tools?
 - Do they have physical access to your machines?

Trusted Computing Base: what you trust

- **Trusted Computing Base**

- Set of systems/components/people that your security depends on
- Security dependency

- You *need* to trust something

- Trusted $\neq$ Trustworthy
- Reflections on Trusting Trust
 - How much do you need to trust?



Trust/Security Boundary & Attack Surface

- **Security Boundary**

- Perimeter around components of same trust level
- Any data or signals coming in from outside is untrusted and potentially malicious

- **Attack Surface**

- Set of interaction points across a security boundary
- Parts of your system handling input from or otherwise interacting with less trusted and potentially malicious entities
 - Port, Inter-procedure Call, API, parser, etc.
- Some *highly sensitive* systems are even “air-gapped” to minimize the attack surface

Threat Modeling - A cautionary note

- Your threat model is your problem scope.
- Attackers ***don't care*** about your threat model.
- Just because an asset or an attacker are outside your threat model, does not mean that they do not exist.
 - It just means that you have decided that you will not address them in your solution.

The Plan

- What is computer security?
- Threat modeling & safety policies
- Review: x86 and GDB

X86: The De Facto Standard

- Extremely popular for desktop computers
- Alternatives
 - ARM: popular on mobile
 - RISC: very simple instructions
- CISC (complex instruction set computing)
 - Over 1000 distinct opcodes (over 6000 variants)
- Variable-length instructions
- Built of many backwards-compatible revisions



Registers

← 32 bits →

EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL
ESI	Some smaller segments of registers have names also		
EDI			
ESP	(stack pointer)		
EBP	(base pointer)		

General
purpose

Instruction syntax

Be comfortable with both!

AT&T syntax

<opcode> <src>, <dst>

```
// In C: ebx = ebx - 16;  
subl $16, %ebx
```

```
// In C: ebx = *eax;  
movl (%eax), %ebx
```

Historically in UNIX

Intel syntax

<opcode> <dst>, <src>

```
// In C: ebx = ebx - 16;  
sub ebx, 16
```

```
// In C: ebx = *eax;  
mov ebx, [eax]
```

Historically in Windows/DOS

Constants preceded by \$

Registers preceded by %

Indirection uses ()

Instruction Suffix

- 'q' is for quad (64-bit)
- 'l' is for long (32-bit)
- 'w' is for word (16-bit)
- 'b' is for byte (8-bit)

x86's powerful addressing modes

Assembly instruction

Meaning in C syntax

`movl (%eax), %ebx`

`ebx = *eax;`

`movl 0x20(%eax), %ebx`

`ebx = *(eax + 0x20);`

`movl 0x20(%eax, %ecx), %ebx`

`ebx = *(eax + ecx + 0x20);`

`movl 0x20(%eax, %ecx*4), %ebx`

`ebx = *(eax + (ecx * 4) + 0x20);`

Question: why do so many addressing modes exist?

Why so many addressing modes?

You can always use arithmetic instructions to compute an address

```
movl 0x20(%ebx, %eax*4), %ebx
```

Could be written as

```
addl $0x20, %ebx  
imull $4, %eax, %eax  
addl %eax, %ebx  
movl (%ebx), %eax
```

But, more instructions $\Rightarrow$
slower

x86 has a lot of the usual instructions

`add/sub/mul/div` – arithmetic

`call` – Jump to a function, after placing a return addr on the stack

`ret` – return to an addr on the stack

`mov` – x86 does loads and stores with a single instruction

- Load: `movl (%eax), %ebx` // In C: `ebx = *eax;`
- Store: `movl %eax, (%ebx)` // In C: `*ebx = eax;`

x86 has many unusual instructions

leal is worth knowing

- Load effective address
- Compute the address in of a mem op but don't access memory

Assembly instruction

Meaning in C syntax

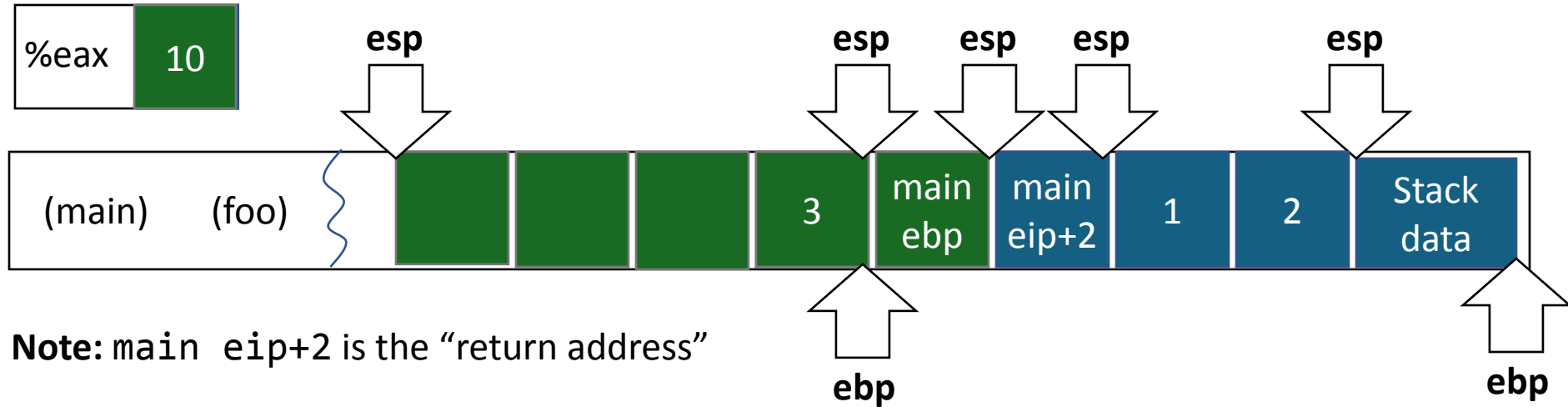
`movl 0x20(%eax, %ecx*4), %ebx`

`ebx = *(eax + (ecx * 4) + 0x20);`

`leal 0x20(%eax, %ecx*4), %ebx`

`ebx = (eax + (ecx * 4) + 0x20);`

Implementing a function call



`main:`

```
...
eip → subl    $8, %esp
eip → movl    $2, 4(%esp)
eip → movl    $1, (%esp)
eip → call    foo
eip → addl    $8, %esp
...
```

`foo:`

```
eip → pushl   %ebp
eip → movl    %esp, %ebp
eip → subl    $16, %esp
eip → movl    $3, -4(%ebp)
eip → movl    8(%ebp), %eax
eip → addl    $9, %eax
eip → leave
eip → ret
```

Function Calls: High level points

Locals are organized into stack frames

- Callees exist at lower address than the caller

On call:

- Save `%eip` (instruction pointer) so you can restore “control” on return
- Save `%ebp` (frame pointer) so you can restore “data frame” on return

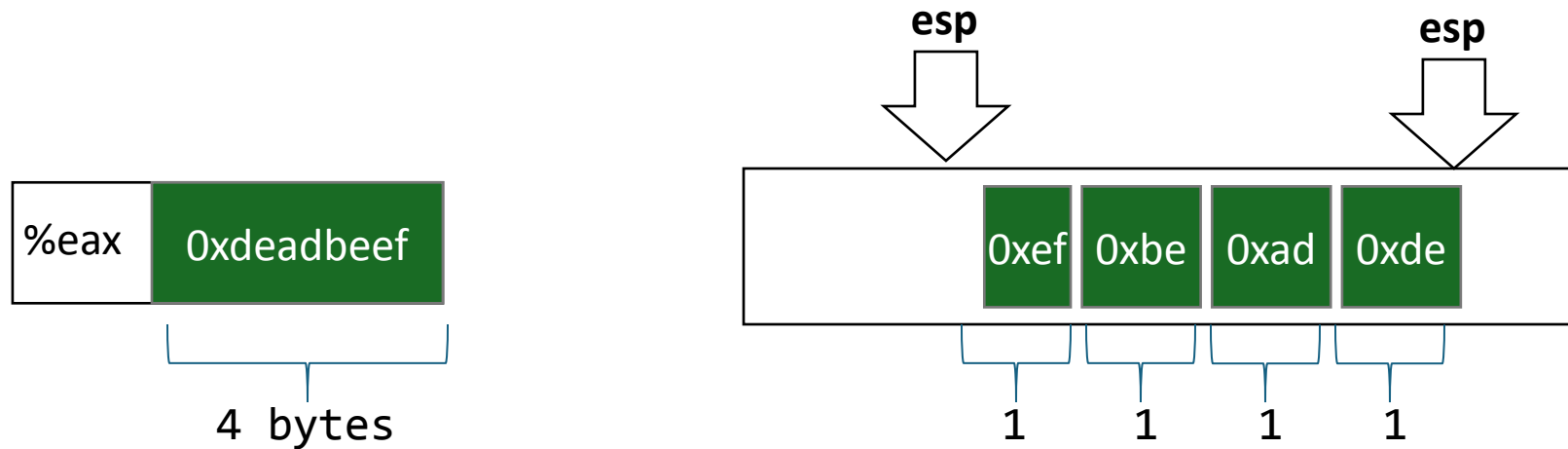
Implementation details are largely by convention

- Somewhat codified by hardware

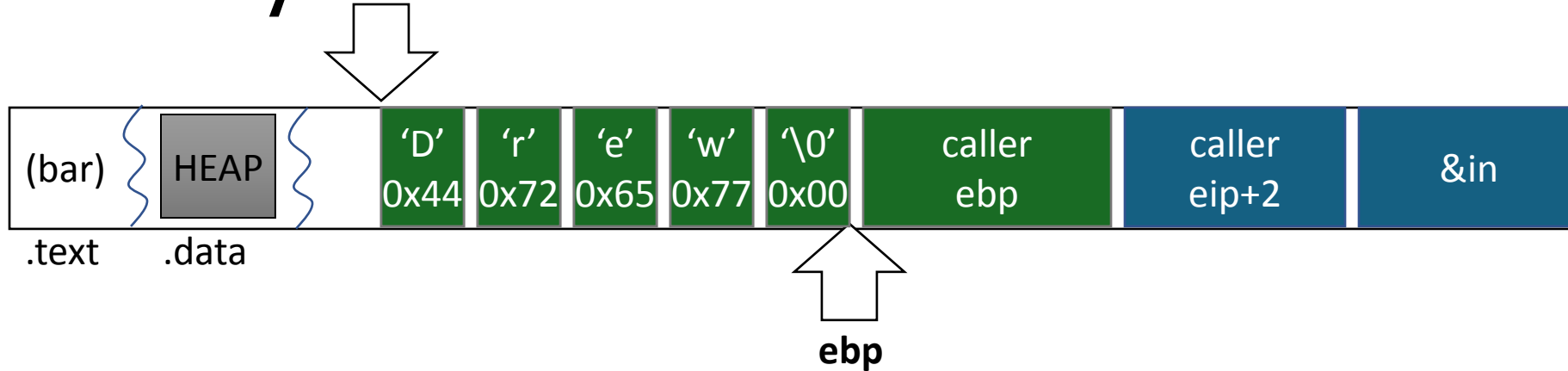
Data types / Endianness

x86 is a little-endian architecture

`pushl %eax`



Arrays^{esp}



```
void bar(char * in){  
    char name[5];  
    strcpy(name, in);  
}
```

```
bar:  
    pushl    %ebp  
    movl     %esp, %ebp  
    subl     $5, %esp  
    movl     8(%ebp), %eax  
    movl     %eax, 4(%esp)  
    leal     -5(%ebp), %eax  
    movl     %eax, (%esp)  
    call     strcpy  
    leave  
    ret
```

You should know all the basic commands

- Run `r`
- Set breakpoints in functions `b foo`
- Continue `c`
- Run line by line of code `step/next`
- See values of variables `print val`
- See backtrace `bt`

Basic idea of GDB: debug your programs

Usual operation

- Set breakpoints in functions
- Run line by line of code
- See values of variables

```
b foo  
step/next  
print val
```

Low-level operation

- Set breakpoints on assembly instructions
- Run instruction by instruction
- See value of memory locations and registers

```
b *0x5655619d  
stepi/nexti  
print *(int *) 0xfffffce8c  
print $eax
```

Have you seen GDB's text user interface

Use this it's much easier

To show code

`layout src`

To show assembly

`layout asm`

To exit tui

`tui disable`

Review

- Two models of security: binary vs. risk management
- To secure a system, you need a threat model
 - Safety Policy - What properties do you care to enforce?
 - Attacker model - Who can attack the system, how?
- 3 core safety policies
 - Confidentiality - Who can read what?
 - Integrity - Who can write what?
 - Availability - Who is allowed to turn off the system.

For next time

- We learn the most basic memory corruption attack: the buffer overflow
- Optional Reading: [Smashing the Stack for Fun and Profit](#)
- Optional Reading: [Reflections on Trusting Trust](#)